

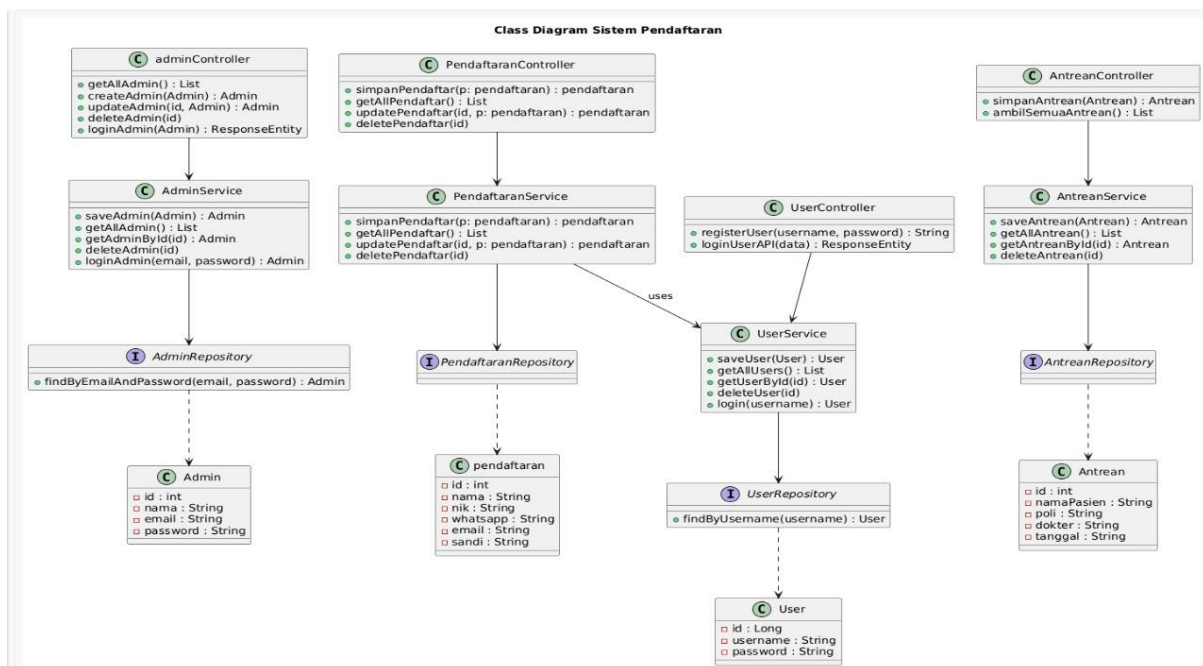
# Class Diagram

1. Anggi Beauty Rahmaputri (41524010061)
2. Devi Dwi Anggraini (41524010081)
3. Nia Putri Rahmadani (41524010124)

## Sistem Pendaftaran Antrean (Antrepbo)

### 1. Skema Class Diagram

Berikut ini adalah skema class diagram dalam sistem Pendaftaran Antrean (Antrepbo) yang mencakup struktur arsitektur backend (Entity, Repository, Service, Controller) menggunakan ekosistem Spring Boot.



Sistem ini terdiri dari empat lapisan utama yang bekerja secara hierarkis:

- Model (Entity): Admin, User, pendaftaran, Antrean
- Repository: AdminRepository, UserRepository, PendaftaranRepository, AntreanRepository
- Service: AdminService, UserService, PendaftaranService, AntreanService
- Controller: adminController, UserController, PendaftaranController, AntreanController

### Hubungan Controller – Service – Repository

| Controller            | Service            | Repository            |
|-----------------------|--------------------|-----------------------|
| adminController       | AdminService       | AdminRepository       |
| UserController        | UserService        | UserRepository        |
| PendaftaranController | PendaftaranService | PendaftaranRepository |
| AntreanController     | AntreanService     | AntreanRepository     |

### Hubungan di dalam Model

Model dalam sistem ini bersifat sederhana dan independen. Tidak ada relasi langsung antar entitas melalui Foreign Key di tingkat objek (seperti @ManyToOne atau @OneToOne). Namun terdapat relasi fungsional implisit: saat data pendaftaran disimpan, sistem secara otomatis mendaftarkan email dan sandi pasien ke entitas User sebagai akun login.

| Model / Entitas | Atribut Utama                         | Keterangan                                       |
|-----------------|---------------------------------------|--------------------------------------------------|
| Admin           | id, nama, email, password             | Pengguna dengan hak akses penuh                  |
| User            | id, username, password                | Akun login pasien (auto-dibuat dari pendaftaran) |
| pendaftaran     | id, nama, nik, whatsapp, email, sandi | Data pendaftaran pasien baru                     |
| Antrean         | id, namaPasien, poli, dokter, tanggal | Data antrean kunjungan dokter                    |

## 2. Penjelasan Lengkap Class Diagram Proyek

Arsitektur kode program di atas mengimplementasikan Layered Architecture (Arsitektur Berlapis) yang dipadukan dengan pola dasar Domain-Driven Component di dalam ekosistem Spring Boot. Pemisahan kelas ke dalam paket-paket (packages) bertujuan untuk memenuhi prinsip Separation of Concerns (SoC) agar setiap lapis memiliki tanggung jawab yang terisolasi dan modular.

Berikut adalah analisis fungsional mendalam untuk masing-masing komponen:

### A. Lapisan Model (Domain Entities)

Lapisan ini berisi representasi objek bisnis (Plain Old Java Object / POJO) yang dipetakan langsung ke tabel database menggunakan anotasi JPA/Hibernate (@Entity, @Table, @Id).

#### 1. Admin

Entitas ini merepresentasikan pengguna dengan hak akses administratif penuh terhadap sistem. Admin dapat mengelola data pendaftar, melihat semua antrean, dan mengelola akun admin lainnya.

- id (int) – Primary key, di-generate otomatis oleh database (IDENTITY strategy).
- nama (String) – Nama lengkap administrator.

- email (String) – Alamat email yang digunakan sebagai identitas unik untuk login.
- password (String) – Kata sandi untuk autentikasi admin.

Anotasi `@Table(name = "admin")` memetakan kelas ini ke tabel "admin" di database H2.

## 2. User

Entitas User merepresentasikan akun login untuk pasien. Secara unik, akun ini tidak dibuat secara manual oleh pasien, melainkan dibuat secara otomatis oleh sistem pada saat pasien mengisi formulir pendaftaran (melalui `PendaftaranService`).

- id (Long) – Primary key bertipe Long untuk mengakomodasi jumlah pengguna yang besar.
- username (String) – Nama pengguna untuk login; diisi dengan nilai email dari data pendaftaran.
- password (String) – Kata sandi login; diisi dengan nilai sandi dari data pendaftaran.

Anotasi `@Table(name = "USERS")` memetakan kelas ini ke tabel "USERS" di database. Perlu dicatat bahwa nama tabel menggunakan huruf kapital semua karena H2 Database bersifat case-sensitive untuk nama tabel.

## 3. pendaftaran

Entitas ini merepresentasikan data lengkap pasien yang mendaftar ke sistem untuk pertama kalinya. Data yang disimpan bersifat lebih kaya dibanding User karena mencakup informasi identitas kependudukan.

- id (int) – Primary key auto-generated.
- nama (String) – Nama lengkap pasien.
- nik (String) – Nomor Induk Kependudukan, identitas resmi pasien.
- whatsapp (String) – Nomor WhatsApp untuk keperluan notifikasi.
- email (String) – Alamat email yang nantinya menjadi username akun User.
- sandi (String) – Kata sandi yang nantinya disalin ke akun User.

Catatan penting: Nama kelas ini ditulis lowercase (pendaftaran) yang merupakan ketidaksesuaian dengan konvensi penamaan kelas Java (PascalCase). Dalam praktik terbaik, seharusnya ditulis sebagai Pendaftaran.

#### 4. Antrean

Entitas ini merepresentasikan satu sesi kunjungan seorang pasien ke poli tertentu. Tidak ada relasi objek langsung ke entitas User atau pendaftaran; sistem menggunakan nama pasien sebagai teks bebas.

- id (int) – Primary key auto-generated.
- namaPasien (String) – Nama pasien yang mengambil antrean.
- poli (String) – Nama poli atau departemen yang dituju (mis. "Poli Umum", "Poli Gigi").
- dokter (String) – Nama dokter yang akan menangani pasien.
- tanggal (String) – Tanggal kunjungan dalam format teks.

### B. Lapisan Repositori (Data Access Layer)

Semua kelas di dalam paket ini berupa antarmuka (interface) yang mewarisi JpaRepository dari Spring Data JPA. Lapisan ini tidak memerlukan penulisan implementasi query SQL manual untuk operasi CRUD standar (Create, Read, Update, Delete) karena sudah disediakan otomatis oleh framework Spring.

#### 1. AdminRepository

Interface ini mewarisi JpaRepository<Admin, Integer>, sehingga tipe entitasnya adalah Admin dan tipe primary key-nya adalah Integer.

Metode kustom yang ditambahkan:

- findByEmailAndPassword(String email, String password) : Admin – Metode ini krusial untuk proses login admin. Spring Data JPA secara otomatis menerjemahkan nama metode ini menjadi query SQL: `SELECT * FROM admin WHERE email = ? AND password = ?`. Ini adalah contoh penggunaan fitur Query Method Derivation dari Spring Data JPA.

## 2. UserRepository

Interface ini mewarisi `JpaRepository<User, Long>`, dengan tipe primary key Long sesuai atribut id di entitas User.

Metode kustom yang ditambahkan:

- `findByUsername(String username) : User` – Digunakan oleh `UserService` untuk mencari akun pengguna berdasarkan username (yang berisi email) pada saat proses login pasien.

## 3. PendaftaranRepository

Interface ini mewarisi `JpaRepository<pendaftaran, Integer>`. Tidak menambahkan metode kustom apapun karena seluruh operasi yang dibutuhkan (simpan, ambil semua, hapus berdasarkan ID, update) sudah tersedia dari `JpaRepository` secara bawaan.

## 4. AntreanRepository

Interface ini mewarisi `JpaRepository<Antrean, Integer>`. Serupa dengan `PendaftaranRepository`, tidak ada metode kustom tambahan. Operasi CRUD standar dari `JpaRepository` sudah mencukupi kebutuhan fungsional pengelolaan antrean.

## C. Lapisan Service (Business Logic Layer)

Paket ini memuat kelas-kelas konkret (bukan interface) tempat seluruh kalkulasi algoritma, logika operasional, aturan keamanan, serta penegakan validasi bisnis berjalan. Setiap kelas Service dianotasi dengan `@Service` agar Spring Boot dapat mengelolanya sebagai komponen yang bisa diinjeksikan (Dependency Injection).

### 1. AdminService

Kelas ini bertanggung jawab atas seluruh logika bisnis yang berkaitan dengan entitas Admin. Ia menyuntikkan (inject) `AdminRepository` sebagai dependensinya.

Metode-metode utamanya:

- `saveAdmin(Admin)` : Admin – Menyimpan atau memperbarui data admin ke database via repository.
- `getAllAdmin()` : `List<Admin>` – Mengambil seluruh data admin dari database.
- `getAdminById(int id)` : Admin – Mencari satu admin berdasarkan ID-nya. Menggunakan `.orElse(null)` untuk menangani kasus admin tidak ditemukan.
- `deleteAdmin(int id)` – Menghapus data admin berdasarkan ID.
- `loginAdmin(String email, String password)` : Admin – Mendelegasikan proses verifikasi login ke `AdminRepository.findByEmailAndPassword()`. Jika admin ditemukan, proses login berhasil; jika null, login gagal.

## 2. UserService

Kelas ini mengelola logika bisnis untuk entitas User. Ia menyuntikkan `UserRepository` sebagai dependensinya.

Metode-metode utamanya:

- `saveUser(User)` : User – Menyimpan data user baru ke database. Metode ini tidak hanya dipanggil oleh `UserController`, tetapi juga dipanggil oleh `PendaftaranService` saat terjadi pendaftaran pasien baru (cross-service dependency).
- `getAllUsers()` : `List<User>` – Mengambil semua data user.
- `getUserById(Long id)` : User – Mencari user berdasarkan ID.
- `deleteUser(Long id)` – Menghapus user berdasarkan ID.
- `login(String username)` : User – Mencari user berdasarkan username (yang berisi email) untuk keperluan autentikasi pasien.

## 3. PendaftaranService

Ini adalah kelas Service dengan logika paling kompleks dalam sistem. Ia menyuntikkan dua dependensi sekaligus: `PendaftaranRepository` DAN `UserService`. Hal ini menciptakan pola cross-service call yang mencerminkan logika bisnis terpadu.

Alur kerja metode `simpanPendaftar()`:

- Aksi 1: Menyimpan data lengkap pendaftar (nama, NIK, whatsapp, email, sandi) ke tabel "pendaftaran" melalui `PendaftaranRepository`.

- Aksi 2: Secara otomatis membuat akun User baru dengan username = email pendaftar dan password = sandi pendaftar, lalu menyimpannya ke tabel "USERS" melalui UserService.

Logika ini menjamin bahwa setiap pasien yang berhasil mendaftar akan langsung memiliki akun untuk login ke sistem, tanpa perlu registrasi ulang.

Metode lain:

- getAllPendaftar() : List<pendaftaran> – Mengambil semua data pendaftar.
- deletePendaftar(int id) – Menghapus data pendaftar berdasarkan ID.
- updatePendaftar(int id, pendaftaran) : pendaftaran – Memperbarui data pendaftar yang sudah ada. Menggunakan pola .map() dari Optional untuk operasi update yang aman.

#### 4. AntreanService

Kelas ini mengelola logika bisnis untuk data antrean. Ia hanya menyuntikkan AntreanRepository sebagai satu-satunya dependensi, menjadikannya kelas Service paling sederhana dalam sistem.

Metode-metode utamanya:

- saveAntrean(Antrean) : Antrean – Mendaftarkan antrean baru ke database.
- getAllAntrean() : List<Antrean> – Mengambil semua riwayat antrean.
- getAntreanById(int id) : Antrean – Mencari antrean berdasarkan ID.
- deleteAntrean(int id) – Menghapus data antrean berdasarkan ID.

#### D. Lapisan Controller (API View Layer)

Bertindak sebagai gerbang komunikasi eksternal aplikasi. Kelas-kelas di layer ini dianotasi dengan @RestController (atau @Controller) dan bertugas menerima HTTP Request dari klien (browser/frontend HTML), melakukan konversi JSON ke Objek Java (melalui @RequestBody), lalu mengembalikan objek Java kembali menjadi JSON dalam wadah ResponseEntity yang memuat HTTP Status Code (seperti 200 OK, 201 Created, 401 Unauthorized). Layer ini dilarang memuat komputasi bisnis langsung.



## 1. adminController

Kelas ini mengelola endpoint REST API untuk entitas Admin. Base URL: /api/admin. Menggunakan @RestController dan @CrossOrigin(origins = "\*") untuk mengizinkan akses dari semua origin (CORS policy).

| HTTP Method | Endpoint         | Fungsi                                   |
|-------------|------------------|------------------------------------------|
| GET         | /api/admin       | Mengambil semua data admin (getAllAdmin) |
| POST        | /api/admin       | Membuat admin baru (createAdmin)         |
| PUT         | /api/admin/{id}  | Memperbarui data admin (updateAdmin)     |
| DELETE      | /api/admin/{id}  | Menghapus admin (deleteAdmin)            |
| POST        | /api/admin/login | Autentikasi login admin → ResponseEntity |

Pada endpoint login, controller mengembalikan ResponseEntity.ok(admin) jika berhasil, atau ResponseEntity.status(401) jika gagal. Logika ini memungkinkan frontend JavaScript menangani respons sukses vs gagal secara berbeda.

## 2. UserController

Kelas ini berbeda dari controller lainnya karena menggunakan @Controller (bukan @RestController), yang berarti beberapa endpoint-nya dapat mengembalikan nama view (template) alih-alih JSON.

| HTTP Method | Endpoint | Fungsi |
|-------------|----------|--------|
|-------------|----------|--------|

|      |                 |                                          |
|------|-----------------|------------------------------------------|
| POST | /register       | Registrasi user (redirect ke login.html) |
| POST | /api/user/login | Login pasien via JSON → ResponseEntity   |

Endpoint `/api/user/login` menerima body JSON berisi email dan password, kemudian memverifikasi dengan mencari user di database berdasarkan email (disimpan di kolom username) dan mencocokkan password secara langsung (plain text comparison).

### 3. PendaftaranController

Kelas ini mengelola endpoint REST API untuk proses pendaftaran pasien baru. Base URL: `/api/daftar`.

| HTTP Method | Endpoint         | Fungsi                                      |
|-------------|------------------|---------------------------------------------|
| POST        | /api/daftar      | Menyimpan pendaftar baru + auto-create User |
| GET         | /api/daftar      | Mengambil semua data pendaftar              |
| PUT         | /api/daftar/{id} | Memperbarui data pendaftar                  |
| DELETE      | /api/daftar/{id} | Menghapus data pendaftar                    |

### 4. AntreanController

Kelas ini mengelola endpoint REST API untuk sistem antrean. Base URL: `/api/antrean`.

| HTTP Method | Endpoint | Fungsi |
|-------------|----------|--------|
|-------------|----------|--------|

|      |                     |                                                     |
|------|---------------------|-----------------------------------------------------|
| POST | /api/antrean/daftar | Mendaftarkan antrean baru<br>(simpanAntrean)        |
| GET  | /api/antrean        | Mengambil semua data antrean<br>(ambilSemuaAntrean) |

### 3. Analisis Pola Hubungan Antar Kelas

Dalam diagram ini, terdapat dua jenis hubungan utama yang membentuk arsitektur sistem:

#### A. Relasi Fungsional Antar Model (Implicit Association)

Berbeda dengan sistem lain yang menggunakan relasi Foreign Key eksplisit di level objek (seperti @ManyToOne), sistem Antrepbo menggunakan relasi data implisit. Hubungan antar model terjadi melalui logika di Service Layer, bukan melalui referensi objek langsung.

Contoh relasi terpenting:

- pendaftaran → User: Saat pendaftaran berhasil disimpan, PendaftaranService secara otomatis membuat objek User baru dengan kredensial yang sama. Ini adalah pola "Side Effect on Save" yang menjamin konsistensi data login.
- Antrean menyimpan namaPasien sebagai teks bebas (String), bukan sebagai referensi objek ke entitas User atau pendaftaran. Ini menyederhanakan desain namun mengorbankan integritas referensial.

#### B. Ketergantungan Aliran Komponen (Dependency Injection)

Menunjukkan prinsip arsitektur Loose Coupling (Ketergantungan Longgar). Kelas di lapis atas selalu bergantung pada kelas di lapis tepat di bawahnya melalui mekanisme Spring @Autowired Dependency Injection:

- adminController @Autowired AdminService
- UserController @Autowired UserService
- PendaftaranController @Autowired PendaftaranService
- AntreanController @Autowired AntreanService
  
- AdminService @Autowired AdminRepository
- UserService @Autowired UserRepository
- PendaftaranService @Autowired PendaftaranRepository + @Autowired UserService

- AntreanService @Autowired AntreanRepository

Pola delegasi searah ini menjamin alur penelusuran kesalahan (debugging) menjadi lebih terprediksi, mempermudah pengujian unit (unit testing), serta menjaga integritas kode program tetap kokoh saat aplikasi dikembangkan dalam skala yang lebih masif.

### C. Cross-Service Dependency (PendaftaranService → UserService)

Salah satu hubungan yang perlu mendapat perhatian khusus adalah PendaftaranService yang tidak hanya menyuntikkan PendaftaranRepository, tetapi juga menyuntikkan UserService. Ini menciptakan pola ketergantungan antar-Service.

Alasan desain ini dipilih: ketika pasien mendaftar, sistem secara atomik perlu melakukan dua hal – menyimpan data pendaftaran DAN membuat akun login. Dengan mendelegasikan pembuatan akun ke UserService (bukan langsung ke UserRepository), sistem menjaga prinsip bahwa setiap Service hanya bertanggung jawab atas logika domain-nya sendiri.

## 4. Ringkasan Struktur Paket Proyek

| Paket (Package) | Kelas/Interface                                                           | Tanggung Jawab                       |
|-----------------|---------------------------------------------------------------------------|--------------------------------------|
| model           | Admin, User, pendaftaran, Antrean                                         | Representasi data / entitas database |
| repository      | AdminRepository, UserRepository, PendaftaranRepository, AntreanRepository | Akses database (CRUD via JPA)        |
| service         | AdminService, UserService, PendaftaranService, AntreanService             | Logika bisnis & validasi             |

|            |                                                                                    |                                     |
|------------|------------------------------------------------------------------------------------|-------------------------------------|
| controller | adminController,<br>UserController,<br>PendaftaranController,<br>AntreanController | Endpoint REST API (HTTP<br>Handler) |
|------------|------------------------------------------------------------------------------------|-------------------------------------|

Struktur paket ini sepenuhnya mengikuti konvensi Spring Boot Standard Layered Architecture yang memisahkan concern secara vertikal (per fitur: Admin, User, Pendaftaran, Antrean) dalam lapisan horizontal (Model–Repository–Service–Controller).